

VIT BHOPAL

PROJECT REPORT

VIT BHOPAL HEALTH CENTRE APPOINTMENT MANAGEMENT SYSTEM

NAME	PRIYANSHU RAJ
REG_NO	26BAI10376
SUBJECT	PYTHON ESSENTIALS
PROJECT TYPE	CONSOLE-BASED PYTHON APPLICATION
YEAR	2026–2027
DATE	28-09-2026

Submitted as a project report for the Python Essentials course.

TABLE OF CONTENTS

1. Introduction	3
2. Problem Statement	4
3. Functional Requirements	5–8
4. Non-functional Requirements	9
5. Design Diagrams	12–17
6. Design Decisions & Rationale	18–19
7. Implementation Details	20–23
8. Screenshots / Results	24–27
9. Testing Approach	28
10. Challenges Faced	29
11. Learning & Key Takeaways	30
12. Future Enhancements	31–32
13. References	33

Report basis

This report is written directly from the supplied Python program and the two supplied console screenshots. Features that are not present in the code—such as login, database storage, claims, or administrator roles—are not presented as current features.

1. Introduction

The VIT Bhopal Health Centre Appointment Management System is a Python-based console application designed to simplify the process of booking and viewing health-centre appointments. The program guides a student through a sequence of menus so that appointment information can be entered in a structured way.

- Collects the student's name, registration number, age and mobile number.
- Displays five available medical departments and their consultation fees.
- Validates the selected department before continuing.
- Accepts an appointment date in DD-MM-YYYY format and prevents past dates.
- Provides six predefined appointment time slots.
- Creates a unique appointment ID beginning with APT1001.
- Displays a confirmation summary after successful booking.
- Allows all appointments stored during the current program run to be viewed.
- Allows users to view department names and consultation fees.
- Uses Python functions, dictionaries, lists, loops, conditional statements, formatted strings and the datetime module.

The current version stores appointments in the in-memory Python list **Appointments**. Therefore, appointments remain available while the program is running but are not saved permanently after the program exits.

2. Problem Statement

Students visiting a campus health centre may need a simple way to select a department, choose a suitable date and time, and receive a clear appointment confirmation. A manual process can lead to incomplete information, invalid dates, confusion about consultation fees, or difficulty keeping track of appointments made during a session.

Major problems addressed by the project:

- Student information may be entered inconsistently.
- Department selection needs a controlled list instead of free-text entry.
- Past dates should not be accepted for new appointments.
- Available time slots should be clearly displayed.
- Consultation fees should be visible before an appointment is finalized.
- The user should receive an appointment ID and a complete confirmation.
- Appointments created during one program session should be viewable from a single menu.

Proposed Solution

The Python program provides a menu-driven workflow for collecting student details, selecting a department, validating an appointment date, choosing a time slot, storing the appointment in a list, and displaying the final appointment summary. It also provides separate options for viewing all current-session appointments and department fees.

3. Functional Requirements — Part 1

1. Main Menu

- Option 1: Book New Appointment
- Option 2: View All Appointments
- Option 3: View Department Fees
- Option 4: Exit the application

2. Student Details

- Name is required and empty input is rejected.
- Registration number is required and empty input is rejected.
- Age is converted to an integer and must be greater than zero.
- Mobile number is converted to an integer and must be greater than zero.

3. Department Selection

The program displays five departments. The user enters a choice from 1 to 5. A loop keeps asking until a valid department key is selected.

Current departments

Choice	Department	Fee
1	General Medicine	₹500
2	Cardiology	₹1000
3	Orthopedics	₹800
4	Pediatrics	₹600
5	Dermatology	₹900

3. Functional Requirements — Part 2

4. Appointment Date

The program reads the current system date using Python's datetime module. The user must enter a date in DD-MM-YYYY format. The date is converted into a datetime.date object. If the entered date is earlier than today, the program rejects it and asks again.

5. Time Slot Selection

Six fixed time slots are displayed. The user enters a number from 1 to 6. The input is checked with **isdigit()** and range validation before the selected slot is returned.

No.	Time Slot
1	9:00 AM
2	10:00 AM
3	11:00 AM
4	2:00 PM
5	3:00 PM
6	4:00 PM

6. Appointment Creation

After all selections are complete, the program creates a dictionary containing appointment ID, patient details, department, date, time and fee. The dictionary is appended to the global **Appointments** list.

3. Functional Requirements — Part 3

7. Appointment ID

The ID is generated with `f"APT{len(Appointments) + 1001}"`. The first appointment therefore receives APT1001, the second APT1002, and so on during the current execution.

8. Appointment Summary

- Patient name
- Age
- Phone number
- Department
- Appointment date
- Appointment time
- Consultation fee
- Appointment ID
- Reminder to arrive 15 minutes before the appointment

9. View All Appointments

If no appointments have been booked, the program prints a message stating that no appointments have been booked yet. Otherwise, it loops through the Appointments list and prints each appointment's ID, student, department, date, time and fee.

10. View Department Fees

The same department-display function used during booking can be accessed directly from the main menu.

3. Functional Requirements — Part 4

11. Exit

Selecting option 4 ends the main loop and prints a thank-you message.

12. Input Validation

- Empty names are rejected.
- Empty registration numbers are rejected.
- Age must be greater than zero.
- Mobile number must be greater than zero.
- Department choice must be one of the dictionary keys 1–5.
- Appointment date must follow DD-MM-YYYY and must not be in the past.
- Time slot choice must be a numeric value from 1–6.
- Invalid main-menu choices return the user to the menu.

Functional workflow summary

Step	Operation	Output
1	Book appointment	Student details collected
2	Select department	Department and fee selected
3	Select date	Valid future/current date
4	Select time	One predefined time slot
5	Store appointment	Appointment dictionary added
6	Confirm	Appointment summary + ID

4. Non-functional Requirements

1. Usability

The application uses simple numbered menus and direct prompts so that a student can follow the process without needing technical knowledge.

2. Reliability

Validation loops reduce common invalid-input cases, while appointment information is stored in a structured dictionary before being appended to the appointments list.

3. Performance

The application operates on small in-memory collections, so menu display and appointment lookup are lightweight for normal classroom or campus-demo usage.

4. Maintainability

The program is divided into focused functions such as `Display_Departments`, `Get_Student_Details`, `Select_Department`, `Select_Time_Slot`, `Get_appointment_Date`, `Book_Appointment` and `View_All_Appointments`.

5. Portability

The project uses standard Python functionality and the `datetime` module. It is suitable for execution from a terminal on systems with a compatible Python 3 installation.

6. Error Handling

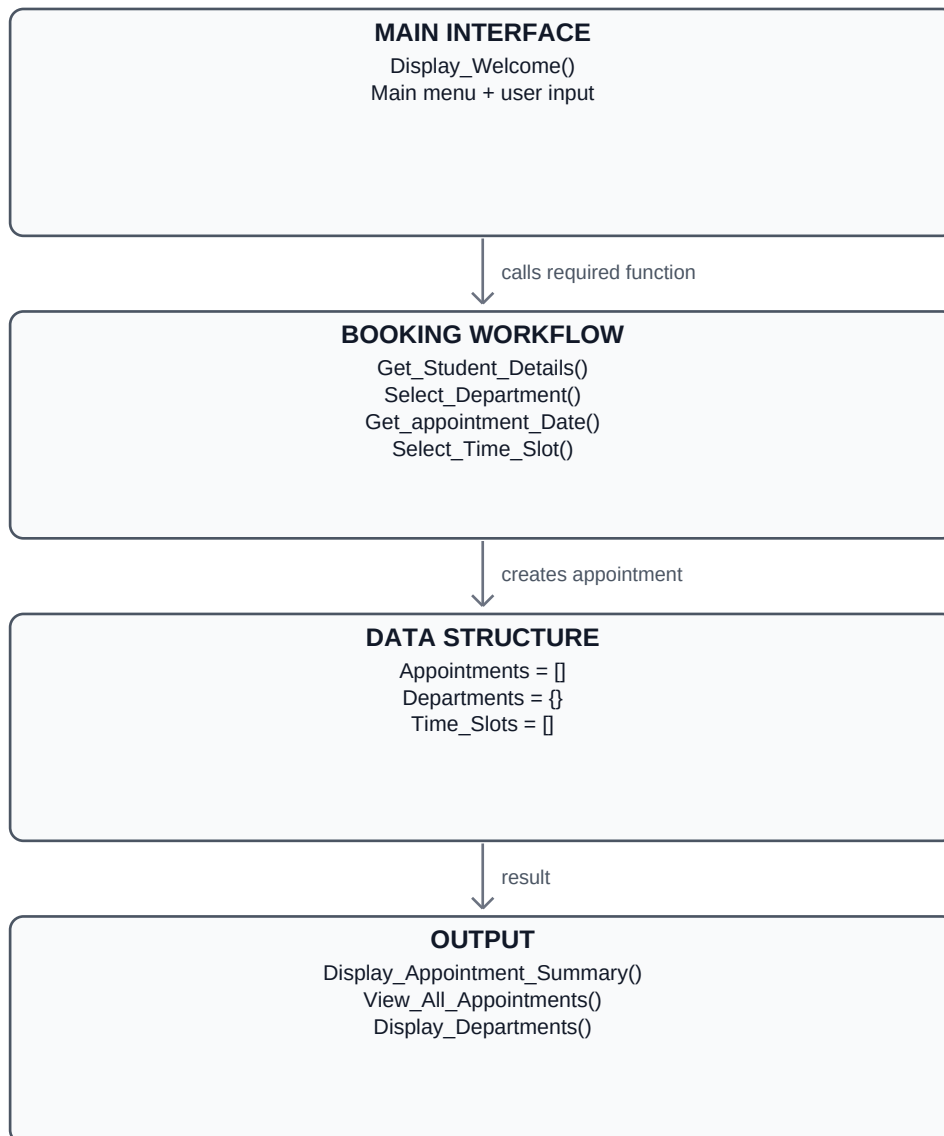
The date function uses `try/except ValueError` to handle invalid dates and invalid date formats. Other menu and selection functions use loops and condition checks to reject invalid choices.

Current limitation

The supplied version does not provide persistent storage, login/authentication, database connectivity, appointment cancellation, or collision checking for two users selecting the same department/date/time.

System Architecture

The application follows a simple function-based console architecture.



Architecture explanation

The Main function controls the program loop. The booking function coordinates the input functions and creates an appointment dictionary. Department data is kept in the Departments dictionary, time slots are kept in the Time_Slots list, and created appointments are kept in the Appointments list.

Architecture Details

Global data structures

- **Departments** — dictionary mapping department numbers to department names and fees.
- **Time_Slots** — list containing six fixed appointment times.
- **Appointments** — list containing appointment dictionaries created during the current run.

Function layer

- `Display_Welcome()` — prints the application heading.
- `Display_Departments()` — prints departments and fees.
- `Get_Student_Details()` — collects and validates student details.
- `Select_Department()` — validates the department choice.
- `Display_Time_Slot()` — prints time slots.
- `Select_Time_Slot()` — validates and returns the selected time.
- `Get_appointment_Date()` — validates the appointment date.
- `Display_Appointment_Summary()` — prints confirmation details.
- `View_All_Appointments()` — prints all stored appointments.
- `Book_Appointment()` — coordinates the complete booking process.
- `Main()` — controls the menu loop.

External storage

There is no database or text-file persistence in the supplied code. The application uses only runtime memory through Python lists and dictionaries.

5. Design Diagrams — Use Case Diagram

Actor: Student / Health Centre User

User action	System response
Open application	Display welcome screen and main menu
Book appointment	Collect student details
Choose department	Display departments and fees
Choose date	Validate DD-MM-YYYY and reject past dates
Choose time	Display six available slots
Confirm booking	Store appointment and print summary
View appointments	Display current-session appointments
View fees	Display all department fees
Exit	Terminate program

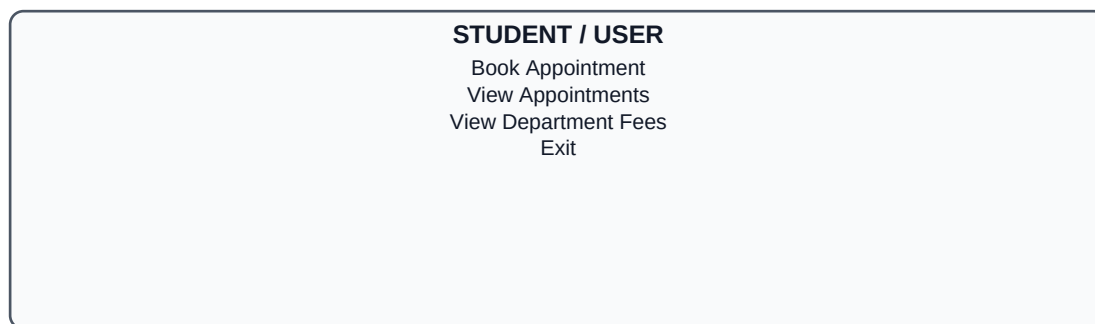
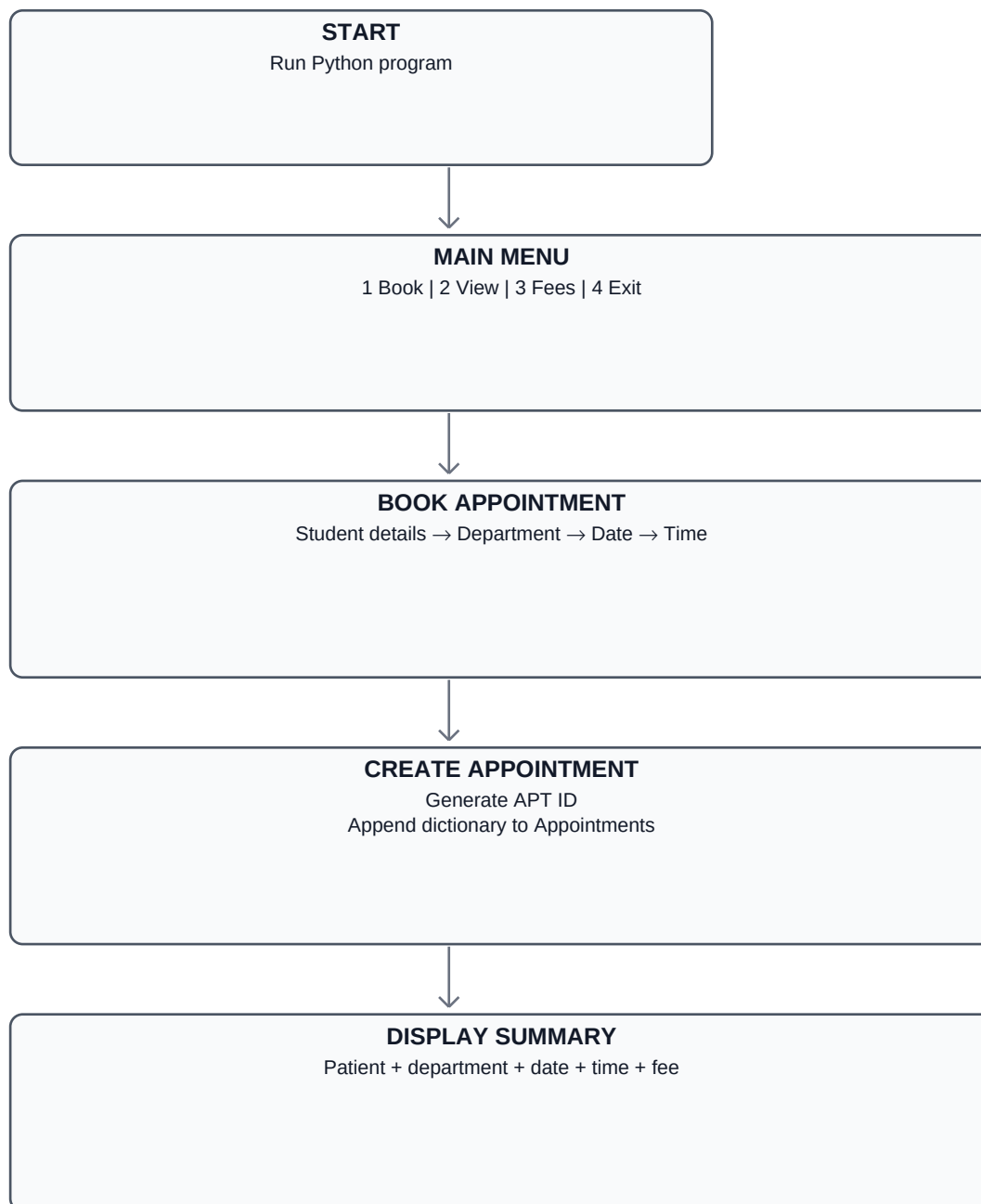


Figure 1: Use case view derived from the actual menu options in the Python program.

Workflow Diagram



Alternative menu branches return to the main menu. Selecting Exit ends the program.

Sequence Diagram — Booking an Appointment

Student	Main	Book_Appointment	Validation Functions
1. Select 1	→	Book_Appointment()	
2. Enter details	→		Get_Student_Details()
3. Select department	→		Select_Department()
4. Enter date	→		Get_appointment_Date()
5. Select time	→		Select_Time_Slot()
6. Confirm data		Create appointment dictionary	
7. Receive result		Display_Appointment_Summary()	

The sequence shows how control moves from the main menu into the booking workflow and then returns to the user with a confirmation summary.

Data Structure Diagram



Figure 4: Runtime data relationship used by the supplied code.

Module / Function Diagram

Function	Responsibility
Display_Welcome	Print application title
Display_Departments	Show department names and fees
Get_Student_Details	Collect and validate student data
Select_Department	Validate department number
Display_Time_Slot	Show available slots
Select_Time_Slot	Validate slot number
Get_appointment_Date	Parse and validate appointment date
Display_Appointment_Summary	Print booking confirmation
View_All_Appointments	Display appointments list
Book_Appointment	Coordinate booking
Main	Run main menu loop

Figure 5: Function-level design of the Python application.

Validation Decision Diagram

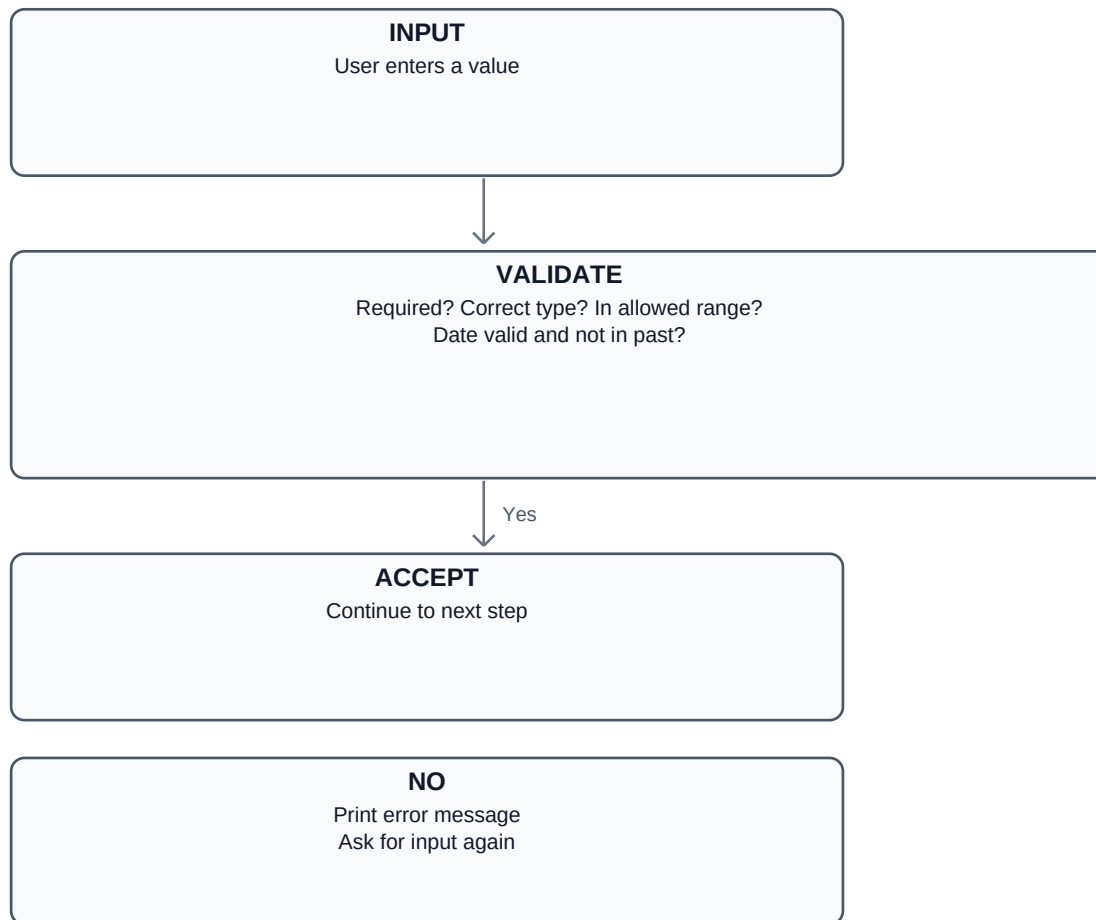


Figure 6: Generic validation loop used throughout the menu-driven application.

6. Design Decisions & Rationale — Part 1

1. Python

Python was selected because it provides concise syntax, built-in collections, input handling, exception handling and the datetime module. These features are sufficient for a small console-based appointment-management project.

2. Console-Based Interface

A console interface keeps the project focused on programming logic, validation, data structures and function decomposition rather than graphical user-interface development.

3. Dictionary for Departments

The Departments dictionary makes it easy to map a numbered choice to a department name and its fee. For example, `Departments["2"]` gives the Cardiology record.

4. List for Time Slots

A simple list is suitable because the six time slots are fixed and ordered. `enumerate()` is used to display the corresponding menu numbers.

5. List for Appointments

Appointments is a list because the program needs to append new appointment records and iterate through them when the user chooses View All Appointments.

Design Decisions & Rationale — Part 2

6. Function-Based Modularity

Each major operation is separated into a function. This reduces repetition and makes the program easier to read, test and modify.

7. Datetime Validation

Using `datetime.date` allows the program to construct a real date and compare it with today's date instead of relying on string comparison.

8. Runtime Appointment IDs

The ID is generated from the current length of the Appointments list. This is simple and adequate for a classroom demonstration, although it is not a permanent database-grade identifier.

9. No Persistent Storage

The supplied code does not use files or a database. This keeps the project lightweight, but all appointments are lost when the program terminates.

10. Validation Loops

while loops are used where the user must keep entering data until it satisfies the required condition. This is particularly visible in department, date and time selection.

7. Implementation Details — Part 1

Imports

```
import datetime
```

The datetime module supplies the current date and the date constructor used for appointment-date validation.

Core data structures

```
Appointments = []
Departments = {
    "1": {"Name": "General Medicine", "Fees": 500},
    "2": {"Name": "Cardiology", "Fees": 1000},
    "3": {"Name": "Orthopedics", "Fees": 800},
    "4": {"Name": "Pediatrics", "Fees": 600},
    "5": {"Name": "Dermatology", "Fees": 900}
}
Time_Slots = ["9:00 AM", "10:00 AM", "11:00 AM",
              "2:00 PM", "3:00 PM", "4:00 PM"]
```

These structures hold all static department/time-slot information and the appointments created during execution.

Implementation Details — Part 2

Student details function

```
def Get_Student_Details():  
    Name = input("Enter Your Name: ").strip()  
    ...  
    Age = int(input("Enter your age: "))  
    ...  
    return {"Name": Name, "Registration Number": Reg_Number,  
            "Age": Age, "Contact Number": Mobile_Number}
```

The function repeatedly asks for required text fields and checks that numeric values are positive.

Department validation

```
Choice = input("\n Select Department(1-5):").strip()  
while Choice not in Departments:  
    print("Invalid Choice! Please select a valid department.")  
    Choice = input("\n Select Department(1-5):").strip()
```

The dictionary keys provide a direct and simple validation mechanism.

Implementation Details — Part 3

Date validation

```
today = datetime.date.today()
day, month, year = date_input.split("-")
apt_date = datetime.date(int(year), int(month), int(day))
if apt_date < today:
    print("Cannot book appointment for past dates!")
    continue
```

The try/except block catches ValueError for malformed dates or invalid numeric date components.

Time validation

```
while not Choice.isdigit() or int(Choice) < 1 or int(Choice) > len(Time_Slots):
    print("Invalid choice! Please select a valid time slot.")
```

This prevents non-numeric values and numbers outside the six available slots.

Appointment construction

```
NewAppointment = {
    "id": f"APT{len(Appointments) + 1001}",
    "patient": Student,
    "department": Departments[Dept_Choice]["Name"],
    "date": Apt_Date,
    "time": Apt_Time,
    "fee": Departments[Dept_Choice]["Fees"]
}
Appointments.append(NewAppointment)
```

Implementation Details — Part 4

Main control loop

```
while True:
    print("\n--- MAIN MENU ---")
    print("1. Book New Appointment")
    print("2. View All Appointments")
    print("3. View Department Fees")
    print("4. Exit")
    Choice = input("\n Enter your Choice (1-4):").strip()
```

The menu dispatches the user's choice to the appropriate function. Choice 4 breaks the loop and exits.

Program entry point

```
if __name__ == "__main__":
    Main()
```

This standard Python entry-point check ensures that Main() runs when the file is executed directly.

Storage model

Unlike the reference Java report, this project has no users.txt, items.txt, claims.txt, database, login module or persistent storage. The current implementation is intentionally a small in-memory appointment system.

8. Screenshots / Results — Screenshot 1

The first supplied screenshot shows the booking flow from the main menu through student details, department selection, date selection and time-slot selection.

```
WELCOME TO VIT BHOPAL HEALTH CENTRE
=====

--- MAIN MENU ---
1. Book New Appointment
2. View All Appointments
3. View Department Fees
4. Exit

Enter your Choice (1-4):1

---STUDENT DETAILS---
Enter Your Name: Priyanshu Raj
Enter your Registration Number: 26BAI10376
Enter your age: 18
Enter your Mobile_Number: 7859064803

---AVAILABLE DEPARTMENTS---
1. General Medicine - Fees:₹500
2. Cardiology - Fees:₹1000
3. Orthopedics - Fees:₹800
4. Pediatrics - Fees:₹600
5. Dermatology - Fees:₹900

Select Department(1-5):2

---SELECT DATE---
Today's Date: 28-09-2026
Enter appointment date (DD-MM-YYYY): 01-10-2026

--- AVAILABLE TIME SLOTS---
1.9:00 AM
2.10:00 AM
3.11:00 AM
4.2:00 PM
5.3:00 PM
6.4:00 PM

---SELECT TIME SLOT--- (1-6):4

=====
APPOINTMENT BOOKED SUCCESSFULLY!
```

Figure 1: Console booking flow — student details, Cardiology selection, date 01-10-2026 and time slot 2:00 PM.

Screenshots / Results — Screenshot 2

The second supplied screenshot shows the successful appointment confirmation generated by the program.

```
priyanshuraj@Guddus-MacBook-Air / % /usr/local/bin/python3.13 /Users/priyanshuraj/vit_bhopu/vityarthi.py

---AVAILABLE DEPARTMENTS---
1. General Medicine - Fees:₹500
2. Cardiology - Fees:₹1000
3. Orthopedics - Fees:₹800
4. Pediatrics - Fees:₹600
5. Dermatology - Fees:₹900

Select Department(1-5):2

---SELECT DATE---
Today's Date: 28-09-2026
Enter appointment date (DD-MM-YYYY): 01-10-2026

--- AVAILABLE TIME SLOTS---
1.9:00 AM
2.10:00 AM
3.11:00 AM
4.2:00 PM
5.3:00 PM
6.4:00 PM

---SELECT TIME SLOT--- (1-6):4

=====
APPOINTMENT BOOKED SUCCESSFULLY!
=====

Patient Name: Priyanshu Raj
Age: 18
Phone: 7859064803

Department: Cardiology
Date: 01-10-2026
Time: 2:00 PM

Consultation Fee: ₹1000
Appointment ID: APT1001

=====
Please arrive 15 minutes before your appointment.
=====
```

Figure 2: Successful booking confirmation showing patient details, Cardiology, date, time, consultation fee and appointment ID APT1001.

Screenshots / Results — Observed Output

The supplied run demonstrates the following successful sequence:

- Main menu option 1 was selected to book a new appointment.
- Student name entered: Priyanshu Raj.
- Registration number entered: 26BAI10376.
- Age entered: 18.
- Mobile number entered in the console.
- Department 2, Cardiology, was selected.
- Appointment date 01-10-2026 was accepted.
- Time slot 4, 2:00 PM, was selected.
- The program printed "APPOINTMENT BOOKED SUCCESSFULLY!" and generated APT1001.

Result interpretation

The screenshots provide direct evidence that the core booking path executes successfully for the shown inputs.

Screenshots / Results — Feature Coverage

Feature	Evidence in supplied code/screenshot
Main menu	Visible in screenshot 1
Student details	Visible in screenshot 1
Department list + fees	Visible in screenshot 1
Date validation workflow	Implemented in code
Time slot list	Visible in screenshot 1
Appointment creation	Implemented in code and shown in screenshot 2
Appointment ID	APT1001 shown in screenshot 2
Consultation fee	■1000 shown in screenshot 2
View all appointments	Implemented in code
View department fees	Implemented in code
Exit	Implemented in code

No additional screenshots were supplied, so this report does not invent screenshots for unshown features.

9. Testing Approach

The application can be tested through functional testing and input-validation testing. The following cases are based on the actual functions in the supplied code.

ID	Test	Input	Expected Result
TC01	Valid student name	Non-empty name	Name accepted
TC02	Empty name	Blank input	Prompt repeated
TC03	Empty registration number	Blank input	Prompt repeated
TC04	Valid age	18	Age accepted
TC05	Non-positive age	0 / negative	Prompt repeated
TC06	Valid department	2	Cardiology selected
TC07	Invalid department	9	Error + retry
TC08	Valid date	01-10-2026	Date accepted if not past
TC09	Past date	Earlier than today	Rejected
TC10	Invalid date format	01/10/2026	Error + retry
TC11	Valid time slot	4	2:00 PM selected
TC12	Invalid time slot	9	Error + retry
TC13	Book appointment	Valid inputs	Appointment stored + summary
TC14	View appointments	After booking	Appointment displayed
TC15	View fees	Menu option 3	Department fees displayed

Observed result from the supplied screenshots: the shown booking scenario completed successfully and produced appointment ID APT1001.

10. Challenges Faced

1. Input Validation

Different inputs require different validation rules. Text fields need non-empty checks, numeric values need positive-value checks, and menu selections need range or key validation.

2. Date Handling

Date input is more complex than ordinary text input because the program must split the string, convert each component to an integer, construct a date object and catch `ValueError`.

3. Coordinating Multiple Functions

The booking workflow depends on several functions returning correct values in the correct sequence. `Book_Appointment()` acts as the coordinator.

4. Appointment ID Generation

The program needs a simple way to generate readable appointment IDs. The current length-based approach is easy to understand for a small runtime-only application.

5. Runtime Storage Limitation

Using a list is simple, but it means data is lost after the program closes. A future version would need a file or database if permanent records are required.

6. User-Friendly Console Output

Clear headings, separators and confirmation details were used so that the user can understand the current step and final result.

11. Learning & Key Takeaways

Technical Learnings

- Using Python functions for modular programming.
- Using dictionaries to represent structured department data.
- Using lists to store ordered values and appointment records.
- Using loops for repeated menus and validation.
- Using if/elif/else for menu control.
- Using f-strings for formatted output.
- Using datetime for date creation and comparison.
- Using try/except for invalid date input.
- Designing a menu-driven console application.
- Creating a simple runtime data model with nested dictionaries.

Practical Learnings

- Breaking a real-world task into smaller functions.
- Thinking about validation before accepting user input.
- Designing a clear user workflow.
- Testing both valid and invalid inputs.
- Reading console output to verify program behavior.
- Documenting the program according to its actual implementation.

Key Takeaway

The main learning from this project is that a useful program needs more than code that runs. The input flow, validation, data organization, output messages and testing all contribute to making the application understandable and dependable.

12. Future Enhancements — Part 1

1. Persistent Storage

Appointments could be stored in JSON, CSV, SQLite or MySQL so that records remain available after the application closes.

2. Appointment Conflict Checking

The system could prevent two appointments from using the same department, date and time slot when the slot capacity is one.

3. Appointment Cancellation and Rescheduling

Students could be allowed to cancel or change an appointment before the scheduled time.

4. Better Mobile Validation

The current mobile number is converted to an integer. A future version could validate length and keep the number as a string to preserve leading zeros.

5. Doctor / Staff Information

Each department could list doctors, consultation rooms and available schedules.

6. Login and Authentication

A future version could provide student and staff accounts with secure authentication.

Future Enhancements — Part 2

7. Graphical User Interface

The console workflow could be converted into a desktop GUI using Tkinter, PyQt or another Python GUI framework.

8. Web or Mobile Application

A web application could allow students to book appointments remotely and allow staff to manage schedules.

9. Notifications

Email or SMS notifications could remind students about upcoming appointments and changes.

10. Doctor Availability

The system could calculate available slots dynamically instead of using one fixed list of six time slots.

11. Database and Reporting

A database-backed version could generate daily, weekly and monthly reports such as department-wise appointment counts.

12. Privacy and Security

A production version should protect personal information, use secure authentication and apply appropriate access controls.

Future architecture

A mature version could evolve from the current single-file console program into separate modules for user management, appointments, doctors, scheduling, storage and reporting.

13. References

1. Python Software Foundation — Python Documentation. Used as the general reference for Python syntax, built-in data structures, functions and exception handling.
2. Python datetime module documentation — reference for date objects, current date retrieval and date comparison.
3. Course/project instructions for the Python Essentials project — used as the academic context for the submitted application.
4. Supplied source code: `vit_yarthi.py` — primary source for the implementation details documented in this report.
5. Supplied console screenshots — primary evidence for the demonstrated appointment-booking run and successful appointment confirmation.

Project Summary

The VIT Bhopal Health Centre Appointment Management System demonstrates how Python can be used to build a practical menu-driven application. The current implementation focuses on appointment booking, department-fee display, date and time selection, input validation, appointment storage during runtime, and confirmation output. The supplied screenshots show a successful Cardiology appointment booking for the demonstrated test case.

END OF REPORT